

Multi-Threaded Port Scanner with Defensive Firewall Detection

1. Project Overview

The Attacker VM hosted the Python-based multi-threaded port scanner, while the Target VM was hardened with iptables rules to detect, log, and ultimately block the scan. The environment specifications are summarized in Table 1 below.

Component	Attacker VM	Target VM
Operating System	Kali Linux	Kali Linux
IP Address	192.168.11.129	192.168.11.130
Role	Scanner / Attacker	Scanned / Defender

The project was structured in three phases: (1) building the multi-threaded port scanner with mutex-protected shared state, (2) configuring iptables detection and prevention rules on the target, and (3) observing the interaction between the scanner and the firewall to draw analytical conclusions from both offensive and defensive perspectives.

2. Offensive Analysis — Multi-Threaded Port Scanner

2.1 Architecture & Threading Model

The scanner accepts a target IP address and port range (1–1024) as inputs. The full range of 1,024 ports is divided equally among 10 concurrent worker threads using ceiling division, ensuring every port is assigned to exactly one thread regardless of whether the range divides evenly.

Each thread calls `socket.connect_ex()` with a 0.5-second timeout on every port in its assigned chunk. `connect_ex()` returns 0 for an open (accepting) port and a non-zero error code for closed or filtered ports, avoiding the exception overhead of `connect()`.

2.2 How Multi-Threading Improves Scanning Speed

Port scanning is inherently an I/O-bound task. The dominant time cost is waiting for TCP handshake responses or timeout expiry — not CPU computation. Sequential scanning forces each port attempt to fully complete before the next begins, yielding a worst-case time of $O(n \times \text{timeout})$. Multi-threading exploits the fact that all 10 threads can simultaneously block on network I/O, reducing the effective time to $O((n / T) \times \text{timeout})$ where T is the thread count.

Scan Method	Worst-Case Time (1–1024 ports, 0.5s timeout)	Speed Gain	Detection Risk
Sequential (1 thread)	~512 seconds	1× (baseline)	Low (slow)
10 Threads (this lab)	~51.5 seconds	~10×	High (rapid)
50 Threads	~10.3 seconds	~50×	Very High

2.3 Evasion Techniques:

A knowledgeable attacker aware of iptables-based detection may employ the following countermeasures:

- **Slow/Low-and-Slow Scanning:** Inserting random delays (e.g., 5–30 seconds between probes) keeps the connection rate below the iptables recent module threshold (10 attempts per 60 seconds in this lab), making the scan appear as sporadic background noise rather than an active sweep.
- **Decoy Scanning (nmap -D):** Forging SYN packets with multiple spoofed source IPs alongside the real attacker IP floods the kernel log with fabricated addresses, obscuring the true origin and complicating attribution analysis.
- **Source IP Rotation:** Routing scan traffic through VPN exit nodes, Tor circuits, or a botnet ensures no single IP accumulates enough hits to trigger rate-based DROP rules.
- **Fragmented Packets:** Splitting TCP SYN packets into tiny IP fragments can cause stateless firewall implementations to miss the destination port field, which is only present in the first fragment.
- **Non-SYN Probe Types:** FIN, NULL, or XMAS scans use non-standard TCP flag combinations that bypass LOG rules specifically matching only SYN-flagged packets, while still eliciting responses from some operating systems.

2.4 Forensic Evidence:

- **Signature Avoidance:** Leaving unique script configurations or custom tool versions can create a "digital fingerprint" that allows incident responders to link multiple disparate attacks to the same source machine.
- **Detection of Tools:** Modern Endpoint Detection and Response (EDR) systems on the attacker's own machine might flag their scanning tools as malicious; clearing evidence helps avoid local security alerts that could disrupt the operation.
- **Chain of Custody:** Forensic evidence like bash history, saved port scan results, or hard-coded target IPs allows investigators to build a chronological case of intent and action.
- **Seizure Evidence:** If an attacker's machine is physically seized by law enforcement, any stored logs, scripts, or results serve as "smoking gun" evidence.

3. Defensive Analysis — iptables Configuration & Observations

3.1 Phase 1: Detection via Logging

The following iptables rule was applied on the Target VM (192.168.11.130) to log all inbound TCP connection attempts from the Attacker VM to ports 1–1024:



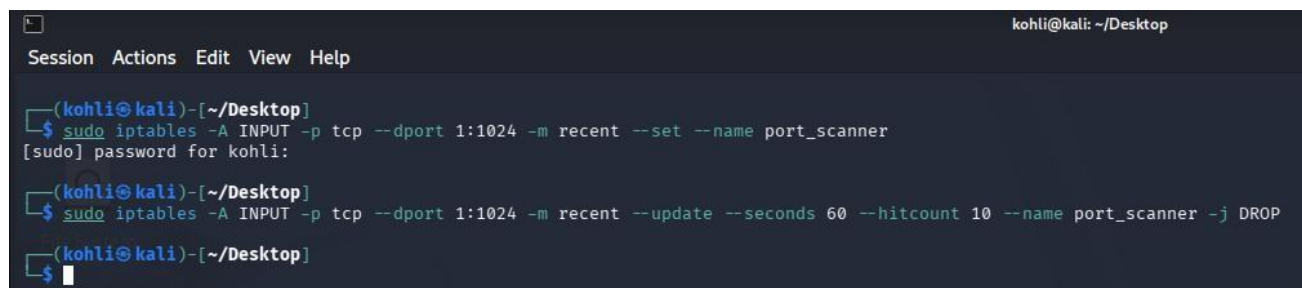
```
Session Actions Edit View Help
(kohli@kali)-[~/Desktop]
$ sudo iptables -F
sudo iptables -A INPUT -s 192.168.11.129 -p tcp --dport 1:1024 -j LOG --log-prefix "SCAN DETECTED: "
```

During this phase, the scanner was executed on the Attacker VM and successfully identified Port 80 (HTTP) as open on the target (an active netcat listener had been started with `nc -l -p 80`). The kernel log exhibited the following patterns that clearly fingerprinted the scan:

- **High-frequency entries:** The `SCAN DETECTED:` prefix appeared dozens of times per second, all sharing the same source IP (192.168.11.129) — a rate impossible to generate through legitimate user activity.
- **Sequential destination ports:** The `DPT=` field in successive log entries cycled through port numbers in thread-interleaved order (e.g., 1, 103, 205, 307 ... interspersed), a pattern exclusive to automated scanning tools.
- **Exclusive SYN flags:** Every logged packet carried only the SYN flag with no accompanying ACK, confirming connection initiation attempts rather than established communications.
- **Absence of ACK/FIN completion:** No subsequent ACK or FIN packets followed the SYN entries, indicating that the scanner was probing rather than maintaining persistent connections.

3.2 Phase 2: Prevention via the recent Module

After confirming detection capability, an advanced two-rule chain was added to automatically DROP packets from the attacker once the connection rate exceeded 10 TCP SYN attempts within a 60-second window:



```
kohli@kali: ~/Desktop
Session Actions Edit View Help
(kohli@kali)-[~/Desktop]
$ sudo iptables -A INPUT -p tcp --dport 1:1024 -m recent --set --name port_scanner
[sudo] password for kohli:
(kohli@kali)-[~/Desktop]
$ sudo iptables -A INPUT -p tcp --dport 1:1024 -m recent --update --seconds 60 --hitcount 10 --name port_scanner -j DROP
(kohli@kali)-[~/Desktop]
$
```

Observed Result

After applying the DROP rule, re-running the port scanner returned "No open ports found." This outcome directly confirms that the firewall was actively silently discarding all TCP SYN packets from the attacker IP before they reached any listening service. The packet counter in `iptables -L -n -v` showed incrementing DROP matches with each scan run, providing forensic evidence that the prevention mechanism was functioning correctly. Port 80, which had been visible in Phase 1, became invisible to the scanner in Phase 2.

3.3 Limitations of iptables-Based Detection

Despite its utility, iptables has several well-documented limitations as a standalone intrusion prevention mechanism:

- **No application-layer visibility (Layer 7 blindness):** iptables operates at OSI Layers 3 and 4. It cannot inspect payload content, TLS certificate fingerprints, or application-layer protocols, leaving it blind to attacks embedded within normal HTTP/HTTPS traffic streams.
- **Low-and-slow scan evasion:** The recent module's hitcount threshold (10 attempts per 60 seconds in this lab) is powerless against a scanner configured to probe only one port every 7 seconds. Such a scan would complete the full 1–1024 range in under 2 hours without ever triggering the DROP rule, yet would remain entirely undetected.

- **No temporal correlation across windows:** iptables rules are memoryless beyond the recent module's fixed time window. An attacker who pauses between bursts resets the counter, and iptables cannot correlate activity across multiple 60-second windows the way a dedicated IDS (Snort, Suricata) would.
- **IP spoofing / address rotation:** Rate-based rules keyed on source IP are trivially bypassed by rotating through multiple source addresses via VPN, Tor, or a botnet, as each new IP starts with a clean counter.
- **Log flooding / disk exhaustion:** Deliberate traffic bursts can saturate /var/log/kern.log, triggering log rotation that permanently discards earlier evidence and generating so much noise that legitimate scan entries become impossible to locate manually.

4. Synchronization — Mutex & Race Condition Analysis

4.1 Why a Mutex is Required

The port scanner's 10 worker threads share a single Python list (`open_ports / results`) into which each thread appends port numbers as it discovers them. Without synchronisation, concurrent append operations from multiple threads racing to modify the list's internal state simultaneously produce a lost-update race condition — a class of concurrency bug that silently discards data without raising any exception.

Although CPython's Global Interpreter Lock (GIL) prevents true simultaneous bytecode execution on multiple CPU cores, the GIL is released between bytecode instructions, and the OS scheduler may context-switch a thread at any point between those instructions. A single `list.append()` call compiles to multiple bytecodes (`LOAD_FAST`, `CALL_METHOD`, `STORE`, etc.), and an interleaved switch between two of these bytecodes in two different threads is sufficient to corrupt the shared list. Explicit locking is therefore mandatory and is industry-standard practice.

4.2 Anatomy of a Race Condition Without the Mutex

The following table illustrates a specific lost-update scenario involving two threads that simultaneously discover open ports and attempt to append without synchronisation:

Step	Thread A	Thread B	results[] State
1	<code>scan_port(80) → True</code>	—	[22]
2	<code>Reads len(results) = 1</code>	<code>scan_port(443) → True</code>	[22]
3	—	<code>Reads len(results) = 1</code>	[22]
4	Appends 80 at index 1	Appends 443 at index 1	[22, 80] ← Port 443 silently LOST!

In Step 4, both threads obtained the same index (1) for their insertion because neither had yet updated the list's internal length field. Thread B's write operation overwrites Thread A's entry at index 1. The final list contains only [22, 80]; Port 443 is permanently lost. The scanner would incorrectly report that Port 443 is closed when in fact it is open — a false negative with potentially serious security implications in a real penetration-testing scenario.

4.3 How the Mutex Prevents the Race Condition

The `threading.Lock()` enforces a mutual exclusion guarantee: at most one thread may hold the lock at any instant. When Thread A enters the `with lock: block`, the lock transitions to the acquired state. If Thread B reaches the same block concurrently, the operating system suspends Thread B at the `lock.acquire()` call until Thread A executes `lock.release()` (performed automatically by the `with` statement's `__exit__` method). Thread B then wakes, acquires the lock, and performs its `append` safely. The two operations are now strictly serialised: they can never interleave, and no data loss is possible.

Performance Note

The mutex introduces negligible overhead in this application. Open ports are rare events (typically fewer than 10–20 per 1,024-port scan), so lock contention is extremely infrequent. The dominant operation — waiting for TCP timeout on closed ports — occurs entirely outside the critical section, meaning all 10 threads spend the overwhelming majority of their execution time running in parallel without blocking one another.
